

Discovery Executive Summary

Project: test-discovery · Generated: 18/08/2026, 19:01:05

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 5 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—
2	Code Quality & Complexity Analysis	—
3	Frontend Modernization Analysis	—
4	Backend Modernization Analysis	—
5	Testing & Quality Assurance Analysis	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a monorepo with a Laravel 12 backend (6 API controllers, 4 Eloquent models, 2 services), a React 19 SPA frontend (4 page components, 4 shared components, 1 custom hook, 1 Zustand store), and a Node/Express dev-API (6 source files). Backend controllers are lean on average (74 LOC) but 4 of 6 access Eloquent models directly — there is no repository layer, and business logic (reachability calculations, tree building) is duplicated across controllers and services in 3 independent locations. The `LegacyReportController` crosses both Discovery and Connect domain boundaries, reading all four models, while `DashboardController` does the same for aggregation. The `LegacyDataMapper` uses PHP `extract()` — a dynamic-variable anti-pattern that creates implicit coupling. On the frontend, all 4 page components and 1 legacy class component make direct `api.*` calls with no service/data-access abstraction layer; the `LegacyMonitorPoller` is a class component with an interval memory leak. The two most urgent risks are (1) the missing repository pattern which scatters ORM access across 32 call sites, and (2) the cross-domain coupling in `LegacyReportController` and `DashboardController` that prevents independent evolution of Discovery and Connect modules. Layers covered: backend (26 PHP files), frontend (15 TS/TSX/JSX files), dev-api (6 JS files).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 LOC avg (max 102)	Good
H2	Missing Service Layer	Controllers accessing repos/models	<15	15–30	>30	25 direct model calls in controllers	Moderate
H3	Missing Repository Pattern	Direct DB access points outside repos	<15	15–30	>30	32 (25 controllers + 7 services)	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (LegacyDataMapper)	Moderate
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% (all Eloquent, no raw SQL)	Good
H7	God Classes	Largest class/file LOC	<500	500–600	>600	276 LOC (dev-api server.js); 222 LOC (ConnectPage.tsx); 165 LOC (MongoService.php)	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	4 (LegacyReportController: 4 cross-domain models; DashboardController: 2)	Moderate

H9	Shared Database Coupling	Tables shared across domains	<30%	30–40%	>40%	~33% — 2 of 4 models queried by cross-domain controllers	Moderate
H10	Duplicated Business Logic (additional)	Independent copies of same algorithm	0	1–2	>2	3 copies of reachability formula + 3 copies of buildTree	Moderate
H11	Missing Frontend Service Layer (additional)	Page components with direct API calls	0	1–3	>3	5 components with 14 total direct api.* calls	Moderate

1.4 Actions Required

Hotspot	Action	Rating	Priority
H3. Missing Repository Pattern	Introduce <code>ConnectMonitorRepository</code> and <code>DiscoveryJobRepository</code> interfaces with Eloquent implementations; bind in <code>AppServiceProvider</code> ; replace all 32 direct <code>Model::</code> calls	High Risk	High
H2. Missing Service Layer	Create <code>ConnectReachabilityService</code> , <code>DiscoveryTreeService</code> , <code>DashboardKpiService</code> ; move business logic out of 4 controllers	Moderate	High
H10. Duplicated Business Logic	Consolidate 3 copies of reachability formula and 3 copies of <code>buildTree()</code> into single canonical service methods	Moderate	High
H8. Domain Boundary Violations	Split <code>LegacyReportController</code> by domain; route cross-domain reads through service interfaces	Moderate	Medium
H9. Shared Database Coupling	Define table ownership per domain; expose read-only service APIs for cross-domain consumers	Moderate	Medium
H5. Shared Utility Abuse	Remove <code>extract()</code> from <code>LegacyDataMapper</code> and <code>LegacyReportController</code> ; move mapping logic to domain service	Moderate	Medium
H11. Missing Frontend Service Layer	Create <code>connectService.ts</code> / <code>discoveryService.ts</code> ; extract domain hooks; convert <code>LegacyMonitorPoller</code> to functional component with cleanup	Moderate	Medium

1.5 Expected Outcomes

- Testability:** Repository interfaces enable unit testing of services without a database; frontend service layer enables mocking API calls in component tests.
 - Single source of truth:** Consolidating the reachability formula and `buildTree()` into canonical service methods eliminates the risk of divergent business logic across endpoints.
 - Independent module evolution:** Enforcing domain boundaries (Discovery vs. Connect) through service interfaces means schema changes in one domain cannot silently break the other.
 - Onboarding velocity:** New contributors can work on Discovery or Connect independently without needing to understand both domains — bounded contexts reduce the knowledge surface area.
 - Migration readiness:** Repository abstraction + domain services position the codebase for future extraction into separate deployable services, should the platform scale beyond a monolith.
- ```
 "stop_reason": "end_turn", "session_id": "a718565e-0393-476a-8617-856b70cd46ec", "total_cost_usd": 2.3541065, "usage": {"input_tokens": 18, "cache_creation_input_tokens": 98918, "cache_read_input_tokens": 1147903, "output_tokens": 31228, "server_tool_use": {"web_search_requests": 0, "web_fetch_requests": 0}, "service_tier": "standard", "cache_creation": {"ephemeral_1h_input_tokens": 98918, "ephemeral_5m_input_tokens": 0, "inference_geo": "not_available", "iterations": [{"input_tokens": 1, "output_tokens": 1999, "cache_read_input_tokens": 106182, "cache_creation_input_tokens": 10409, "cache_creation": {"ephemeral_5m_input_tokens": 0, "ephemeral_1h_input_tokens": 10409, "type": "message"}], "speed": "standard"}, "modelUsage": {"claude-haiku-4-5-20251001": {"inputTokens": 10105, "outputTokens": 16, "cacheReadInputTokens": 0, "cacheCreationInputTokens": 0, "webSearchRequests": 0, "costUSD": 0.010185, "contextWindow": 200000, "maxOutputTokens": 3200}, "opus-4-6": {"inputTokens": 18, "outputTokens": 31228, "cacheReadInputTokens": 1147903, "cacheCreationInputTokens": 98918, "webSearchRequests": 0, "costUSD": 2.3439215, "contextWindow": 200000, "maxOutput": [{"terminal_reason": "completed", "fast_mode_state": "off", "uuid": "f330f0c5-bea8-4977-88b2-fd4b9b4d5a8e"}]}
```

2. Code Quality & Complexity Analysis

EXECUTIVE SUMMARY

The Klearcom platform codebase is compact (3,000 LOC across 50 files) and largely well structured, with no function or method exceeding a cyclomatic complexity of 20. However, one frontend component — `ConnectPage.tsx` — exceeds the 200 LOC single function threshold at 210 source lines, concentrating form state, three TanStack Query hooks, event handlers, and two data tables in a single render function. Business logic duplication sits at the Moderate boundary (5 %): the `buildTree` algorithm is copy-pasted across `DiscoveryController`, `LegacyReportController`, and the dev-API `store.js`, while the reachability-rate calculation is repeated in `RealTimeTestService`, `ConnectController`, and the dev-API `realtime.js` — the latter already flagged inline as a duplicate. Refactoring these two areas will yield the highest return on code quality.

2.1 Benchmark Ratings Summary

| # | Hotspot | Primary KPI | <span class="rating rating-good">Good | <span class="rating rating-moderate">Moderate | <span class="rating rating-high-risk">High Risk | Measured | Rating |
|---|---------|-------------|---------------------------------------|-----------------------------------------------|-------------------------------------------------|----------|--------|
|---|---------|-------------|---------------------------------------|-----------------------------------------------|-------------------------------------------------|----------|--------|

|    |                            |                             |      |         |      |                                                                                     |                                                 |
|----|----------------------------|-----------------------------|------|---------|------|-------------------------------------------------------------------------------------|-------------------------------------------------|
| H1 | High Cyclomatic Complexity | Methods with complexity >20 | <30  | 30–40   | >40  | 0 methods >20 (highest ~15 in ConnectPage.tsx; backend methods all <10)             | <span class="rating rating-good">Good           |
| H2 | Large Functions            | Largest function LOC        | <100 | 100–200 | >200 | ~210 LOC (ConnectPage.tsx); DiscoveryPage.tsx ~166 LOC; all backend methods <60 LOC | <span class="rating rating-high-risk">High Risk |
| H3 | Business Logic Duplication | Duplicated business logic % | <5%  | 5–10%   | >10% | ~5% (buildTree ×3, reachability calc ×3, document serialize ×2)                     | <span class="rating rating-moderate">Moderate   |

No additional hotspots beyond the standard set were observed.

Hotspot Score breakdown

| Component                  | Weight | Sub-score (0–100) | Weighted |
|----------------------------|--------|-------------------|----------|
| Cyclomatic Complexity      | 50%    | 20                | 10.0     |
| Function Size              | 30%    | 70                | 21.0     |
| Business Logic Duplication | 20%    | 38                | 7.6      |
| Hotspot Score              | 100%   |                   | 39 / 100 |

2.4 Actions Required

| Hotspot                         | Action                                                                                                                                                                                                                                                                                                                     | Rating                                          | Priority                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------|
| H2 — Large Functions            | Decompose <code>ConnectPage.tsx</code> (210 LOC) into sub-components ( <code>MonitorForm</code> , <code>MonitorTable</code> , <code>CheckHistoryPanel</code> , <code>TranscriptsPanel</code> ) and custom hooks; apply same decomposition to <code>DiscoveryPage.tsx</code> (166 LOC)                                      | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| H3 — Business Logic Duplication | Consolidate <code>buildTree</code> into a <code>TreeService</code> or model static method; extract reachability calculation into <code>ReachabilityService</code> or <code>ConnectMonitor::computeReachability()</code> ; remove inline copies from <code>ConnectController</code> and <code>LegacyReportController</code> | <span class="rating rating-moderate">Moderate   | <span class="sev sev-high">High         |

2.5 Expected Outcomes

- Lower defect risk on monitor status:** A single reachability-calculation service eliminates the risk of divergent alert thresholds across three code paths (RealTimeTestService, ConnectController, dev-API realtime.js).
  - Faster code reviews:** Splitting `ConnectPage` (210 LOC) and `DiscoveryPage` (166 LOC) into focused sub-components (each <80 LOC) reduces review scope and merge-conflict surface.
  - Safer refactors:** Removing the duplicated `buildTree` copies means IVR tree changes propagate automatically to both the discovery and legacy-report endpoints.
  - Easier onboarding:** New developers can understand each sub-component in isolation instead of tracing a 210-line function with three query hooks, two tables, and conditional rendering.
  - Improved testability:** Extracted hooks ( `useConnectQueries` , `useMonitorForm` ) and services ( `ReachabilityService` , `TreeService` ) can be unit-tested independently without rendering full page components.
- ```

{
  "stop_reason": "end_turn",
  "session_id": "aa15eb01-e01f-4705-b891-5388ccb38547",
  "total_cost_usd": 2.1763269999999997,
  "usage": {
    "input_tokens": 17,
    "cache_creation_input_tokens": 105085,
    "cache_read_input_tokens": 915782,
    "output_tokens": 26367,
    "server_tool_use": {
      "web_search_requests": 0,
      "web_fetch_requests": 0,
      "service_tier": "standard",
      "cache_creation": {
        "ephemeral_1h_input_tokens": 105085,
        "ephemeral_5m_input_tokens": 0,
        "inference_geo": "not_available",
        "iterations": [
          {
            "input_tokens": 1,
            "output_tokens": 1488,
            "cache_read_input_tokens": 99265,
            "cache_creation_input_tokens": 5820,
            "cache_creation": {
              "ephemeral_5m_input_tokens": 0,
              "ephemeral_1h_input_tokens": 5820,
              "type": "message"
            },
            "speed": "standard",
            "modelUsage": {
              "claude-haiku-4-5-20251001": {
                "inputTokens": 8241,
                "outputTokens": 17,
                "cacheReadInputTokens": 0,
                "cacheCreationInputTokens": 0,
                "webSearchRequests": 0,
                "costUSD": 0.008326,
                "contextWindow": 200000,
                "maxOutputTokens": 32000,
                "opus-4-6": {
                  "inputTokens": 17,
                  "outputTokens": 26367,
                  "cacheReadInputTokens": 915782,
                  "cacheCreationInputTokens": 105085,
                  "webSearchRequests": 0,
                  "costUSD": 2.1680009999999994,
                  "contextWindow": 200000,
                  "terminal_reason": "completed",
                  "fast_mode_state": "off",
                  "uuid": "de2d42c6-61ab-49df-8751-a02926e32c63"
                }
              }
            }
          }
        ]
      }
    }
  }
}
```

3. Frontend Modernization Analysis

EXECUTIVE SUMMARY

The Klearcom frontend is a compact React 19 + TypeScript single-page application consisting of 13 source files and 11 component definitions, built with a modern Vite toolchain and well-chosen library stack (Zustand for client state, React Query for server cache). The most severe gap is **significant structural and code duplication** between the two primary feature pages — `ConnectPage` and `DiscoveryPage` — which share near-identical form layouts, query patterns, invalidation logic, and MongoDB transcript rendering blocks that should be extracted into shared components and hooks. A legacy class-based component (`LegacyMonitorPoller.jsx`) with an **uncleared interval leak** and a `LegacyDashboardWidget` that **throws unhandled errors without an Error Boundary** represent resource safety and crash-resilience risks. Component sizes are well within healthy limits and state management is clean and scoped.

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	~33% (3 of 9 component files)	High Risk
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	91% (10 of 11 definitions)	Good
H3	Massive Components	Largest component LOC	<400	400–500	>500	222 LOC (ConnectPage.tsx)	Good
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	18% (2 of 11 components)	Good
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	1–2 levels	Good
H6	Missing Error Boundaries (additional)	Error boundary coverage (%; target 100% of throw-capable subtrees)	>80%	50–80%	<50%	0% (no ErrorBoundary in codebase)	High Risk
H7	Resource / Memory Leak (additional)	Components with uncleared timers/subscriptions (count; target 0)	0	1	>1	1 (LegacyMonitorPoller)	Moderate

3.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — UI Component Duplication	Extract shared <code>TranscriptList</code> component, create <code>useModulePage</code> hook to deduplicate page structure, delete <code>LegacyDashboardWidget.tsx</code>	High Risk	Critical
H6 — Missing Error Boundaries	Add <code>ErrorBoundary</code> at app root and per-route level; replace <code>throw new Error</code> with rendered error state	High Risk	High
H7 — Resource / Memory Leak	Convert <code>LegacyMonitorPoller</code> to functional component with <code>useQuery</code> <code>refetchInterval</code> , or add <code>componentWillUnmount</code> cleanup	Moderate	Medium

3.5 Expected Outcomes

- Shared component library reduces duplication:** Extracting `TranscriptList` and `ModulePageLayout` eliminates ~33% structural duplication and makes adding new feature modules a configuration exercise rather than a copy-paste operation.
- Error Boundaries improve resilience:** Per-route boundaries ensure a failing API in one module cannot crash the entire platform — critical for a voice monitoring dashboard that must stay available.
- Functional component conversion improves safety:** Replacing the class-based `LegacyMonitorPoller` with a `useQuery`-based hook eliminates the interval memory leak and aligns with the codebase's established React Query patterns.
- Deletion of legacy widget reduces maintenance surface:** Removing `LegacyDashboardWidget` eliminates redundant network calls and a throw-without-boundary crash vector.
- Consistent patterns accelerate onboarding:** A single page layout pattern with hooks-based data fetching gives new contributors one clear way to build feature pages.
{"stop_reason":"","end_turn","session_id":"","4e19afb8-a3d4-48ce-80a3-b70d97cda1d5","total_cost_usd":1.7082904999999997,"usage":{"input_tokens":16,"cache_creation_input_tokens":85918,"cache_read_input_tokens":699709,"output_tokens":19670,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":85918,"ephemeral_5m_input_tokens":0},"inference_geo":{"not_available"},"iterations":{"input_tokens":1,"output_tokens":1352,"cache_read_input_tokens":80268,"cache_creation_input_tokens":5650,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":5650},"type":"message"}},{"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":7346,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.007426,"contextWindow":200000,"maxOutputTokens":32000,"opus-4-6":{"inputTokens":16,"outputTokens":19670,"cacheReadInputTokens":699709,"cacheCreationInputTokens":85918,"webSearchRequests":0,"costUSD":1.7008644999999996,"contextWindow":200000,"n[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"6d536e0c-debc-4f30-ae31-da7543f840bc"}}

4. Backend Modernization Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a monorepo with two parallel backend stacks: a PHP 8.3 / Laravel 12 application serving the production API (6 controllers, 19 endpoints, MariaDB + MongoDB) and a Node.js / Express 4.21 dev-api mirroring every endpoint with an in-memory store for local development (18 endpoints). Three occurrences of PHP `extract()` were found — two in `LegacyDataMapper` and one in `LegacyReportController` — where raw request data or untrusted arrays are unpacked into local variables without validation. The codebase has no OpenAPI specification, no API versioning, no contract tests, and no API linting despite exposing a significant REST surface. The dev-api duplicates every Laravel endpoint without a shared contract, which means the two backends can diverge silently — and already have (a `bulk-import` endpoint exists only in dev-api, and the `checks` response shape differs). Overall, the backend carries moderate risk driven by the absence of API governance across both stacks.

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	3	Moderate
H2	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	~49% (18 of 37 endpoints duplicated across two backends with no shared contract; 1 dev-only endpoint with no production equivalent)	High Risk
H3	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0% (no OpenAPI spec, no versioning, no contract tests, no API linting)	High Risk

No additional hotspots beyond the standard set were observed.

4.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Dynamic Variable Creation	Replace 3 <code>extract()</code> calls with explicit variable assignment / typed DTOs; add CI rule to block future <code>extract()</code> usage	Moderate	High
H2 — API Sprawl	Author shared OpenAPI 3.1 spec; reconcile divergent endpoints (<code>bulk-import</code> , <code>checks</code> response shape) between Laravel and Express backends	High Risk	Medium
H3 — Missing API Governance	Introduce OpenAPI spec, API versioning (<code>/api/v1/</code>), Spectral linting, contract tests, <code>throttle</code> middleware, and tighten CORS	High Risk	Medium

4.5 Expected Outcomes

- Replacing `extract()` with explicit FormRequest validation and typed DTOs eliminates untraceable dynamic variable flow and closes the variable-shadowing injection vector in the public `carrierSummary` endpoint.
- A single OpenAPI 3.1 specification shared between Laravel and Express backends ensures both stacks serve identical contracts, enabling auto-generated TypeScript types for the frontend.
- API versioning (`/api/v1/`) provides a safe migration path for breaking changes without disrupting existing consumers.
- Contract tests (Spectral + Prism) running in CI catch endpoint divergence between the dual backends before code merges, preventing the silent drift already observed with `bulk-import` and `checks` .
- Rate limiting (`throttle` middleware) and tightened CORS (`allowed_origins` restricted to known frontend domains) reduce abuse surface on the currently wide-open API.

```
{"stop_reason":"end_turn","session_id":"12d12adf-073b-49db-9276-b352e8d7ba5d","total_cost_usd":1.9716114999999999,"usage":{"input_tokens":92,"cache_creation_input_tokens":97783,"cache_read_input_tokens":1088685,"output_tokens":17668,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":97783,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":{"input_tokens":1,"output_tokens":1158,"cache_read_input_tokens":92449,"cache_creation_input_tokens":5334,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":5334},"type":"message"}},{"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":7199,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.007279,"contextWindow":200000,"maxOutputTokens":32000,"opus-4-6":{"inputTokens":92,"outputTokens":17668,"cacheReadInputTokens":1088685,"cacheCreationInputTokens":97783,"webSearchRequests":0,"costUSD":1.9643324999999998,"contextWindow":200000,"terminal_reason":"completed","fast_mode_state":"off","uuid":"e10e9ce7-98f7-4e43-b050-9e76849c66c3}}
```

5. Testing & Quality Assurance Analysis

EXECUTIVE SUMMARY

The Klearcom platform has a critically thin test suite. The backend (Laravel 12 / PHP 8.3) ships with PHPUnit 11 configured but contains only 2 test files with 3 test methods — none of which exercise any real application code; they assert hardcoded arrays and random integers. All 6 API controllers, both service classes (`RealTimeTestService`, `MongoService`), the `LegacyDataMapper`, and all 4 Eloquent models are completely untested. The frontend (React 19 / TypeScript / Vite) has zero test infrastructure — no Vitest, Jest, Testing Library, Cypress, or Playwright is installed, and zero test files exist across 14 source files. The Node.js dev-api (Express + MongoDB) likewise has zero tests and no test framework. CI runs `vendor/bin/phpunit` on the backend and `npm run build` on the frontend, but the frontend job only type-checks and builds with no test step at all. Estimated overall coverage is below 5%.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	7	High Risk
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	<5% (estimated, test-file-to-source ratio 2/35)	High Risk
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0% (no integration/feature tests exist)	High Risk
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% (0 of 15 endpoints)	High Risk
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	1 (non-deterministic <code>random_int</code> test)	Moderate
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	Backend: runs, not required; Frontend: no test run	Moderate
H7	No Frontend Test Infrastructure (additional)	Frontend test framework installed and tests present	Framework + tests	Framework, no tests	No framework	No framework installed, 0 test files	High Risk
H8	No Dev-API Tests (additional)	Dev-API test-file-to-source ratio	>50%	10–50%	<10%	0% (0 of 6 source files have tests)	High Risk
H9	Assertion-Free / Trivial Tests (additional)	Tests with no meaningful assertions	0	1–2	>2	2 (both existing test files assert only hardcoded values)	Moderate

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Untested Critical Logic	Add PHPUnit unit tests for <code>RealTimeTestService</code> , <code>MongoService</code> , <code>LegacyDataMapper</code> , and all 6 API controllers — prioritize state transition and reachability calculation logic	High Risk	Critical
H2 — Low Test Coverage	Enable PHPUnit coverage collection in CI; install Vitest in frontend and dev-api; target 50% in sprint 1, 75–80% before any major refactor	High Risk	Critical
H3 — Missing Integration Tests	Create <code>tests/Feature/</code> directory; add Laravel HTTP tests using <code>RefreshDatabase</code> against CI MariaDB; add MongoDB integration tests	High Risk	Critical
H4 — Missing Contract Tests	Add JSON Schema or snapshot contract tests for all 15 API endpoints; validate SSE event payloads match frontend type definitions	High Risk	Critical
H5 — Flaky / Skipped Tests	Replace <code>random_int()</code> test with deterministic test exercising real reachability calculation	Moderate	Medium
H6 — No CI Test Gate	Add <code>npm test</code> to frontend CI; add dev-api CI job; configure branch protection requiring all checks to pass	Moderate	High
H7 — No Frontend Test Infrastructure	Install Vitest + Testing Library + jest-dom; add <code>test</code> script; write tests for <code>useRealtimeTest</code> , <code>api/client.ts</code> , and page components	High Risk	Critical
H8 — No Dev-API Tests	Add Vitest + supertest to dev-api; test reachability logic and API contracts to prevent backend/dev-api divergence	High Risk	High
H9 — Assertion-Free Tests	Replace both trivial test files with tests that import and exercise real application classes	Moderate	Medium

5.5 Expected Outcomes

- Critical paths protected:** Unit tests for `RealTimeTestService` and `MongoService` prevent regressions in IVR discovery, TFN reachability, and real-time streaming before they reach production.
 - Refactoring enabled safely:** Reaching 75–80% coverage provides the safety net needed to extract duplicated `buildTree()` logic, eliminate `extract()` usage, and modernize the `LegacyReportController`.
 - CI catches regressions automatically:** Adding test execution to all three CI jobs (backend, frontend, dev-api) with required status checks means broken code cannot merge.
 - Contract stability guaranteed:** Contract tests for the 15 API endpoints ensure that backend changes cannot silently break the frontend and that the dev-api stays in sync with production.
 - Developer confidence restored:** Replacing trivial/flaky tests with meaningful assertions provides an honest signal — green means the application works, red means it doesn't.
- ```

{
 "stop_reason": "end_turn",
 "session_id": "91232d27-f403-4b5b-8254-08776bfe7e18",
 "total_cost_usd": 2.2005325,
 "usage": {
 "input_tokens": 22,
 "cache_creation_input_tokens": 98355,
 "cache_read_input_tokens": 1159263,
 "output_tokens": 25176,
 "server_tool_use": {
 "web_search_requests": 0,
 "web_fetch_requests": 0,
 "service_tier": "standard",
 "cache_creation": {
 "ephemeral_1h_input_tokens": 98355,
 "ephemeral_5m_input_tokens": 0,
 "inference_geo": "not_available",
 "iterations": [
 {
 "input_tokens": 1,
 "output_tokens": 1950,
 "cache_read_input_tokens": 97909,
 "cache_creation_input_tokens": 446,
 "cache_creation": {
 "ephemeral_5m_input_tokens": 0,
 "ephemeral_1h_input_tokens": 446,
 "type": "message"
 }
 }
],
 "speed": "standard",
 "modelUsage": {
 "inputTokens": 7756,
 "outputTokens": 17,
 "cacheReadInputTokens": 0,
 "cacheCreationInputTokens": 0,
 "webSearchRequests": 0,
 "costUSD": 0.007841,
 "contextWindow": 200000,
 "maxOutputTokens": 32000,
 "opus-4-6":

```

```
{ "inputTokens":22,"outputTokens":25176,"cacheReadInputTokens":1159263,"cacheCreationInputTokens":98355,"webSearchRequests":0,"costUSD":2.1926915,"contextWindow":200000,"maxOutput":
[], "terminal_reason":"completed", "fast_mode_state":"off", "uuid":"6b31ae44-04d1-4edd-853d-eb565845f641" }
```